COMPSCI 701:
Creating Maintainable Software

Lecture 5.2: Functions and Maintainability II

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Admin
  - ??
- Test 1
- More Functions and Maintainability
- Class Exercise

Test 1

- When: Friday 29 August 1800-2000
- Location: 401-401, Eng1401 Lecture Theatre OR 401-439, Eng1439 Lecture Theatre.
- Duration: The test is designed to be 1 hour long. The 2 hour period is to allow for setting up and to provide flexibility
- Format: Multiple Choice Questions
- Content: Everything presented up to (and including) Wednesday 27 August
- Support: Restricted Book (Learning Journal)
- 2024 test will eventually be available on Canvas

Previously in COMPSCI701 (Martin on Functions)

- Small
- Do One Thing (“cohesion”)
- One Level of Abstraction per Function
- Reading Code from Top to Bottom
- Use Descriptive Names
- Function Arguments
- Have No Side Effects

These heuristics are not necessarily independent — they all can affect each other

Evaluating Martin (Small!)

- Martin's heuristics are based on his (extensive and valuable) experience — how often do they improve comprehensibility?
- For example, Martin says of the size of functions:

> The first rule of functions is they should be small.
>
> The second rule of functions is that *they should be smaller than that*

- (referring to why functions should be small)

> This is not an assertion that I can justify. I can't provide any references to research that shows that very small functions are better. What I can tell you is that for nearly four decades I have written functions of all different sizes. [...] What this experience has taught me, through long trial and error, is that functions should be very small

- Martin suggests not more than 4 lines! (p107)
- Research has shown that a human's short-term memory only holds a limited amount of information
- $\Rightarrow$ the less that needs to be remembered to understand something, the easier it will be to do so

Where to find small functions

- A large function cannot be just turned into a small function
- A large function is decomposed into several smaller functions
  - E.g. `NoughtsAndCrosses-SFV` versus `NoughtsAndCrosses-MFV`, Discussion 5.1
- Is a single large function easier or harder to understand than several smaller functions?
- With multiple functions, a question than then arises is what order should the functions be presented?

Reading Code From Top to Bottom (Order of Functions)

- Newspaper metaphor “inverted pyramid” (but not explained that way by Martin)
- The first function is the most abstract—essentially everything in it is calls to other, more concrete, functions
- To gain sufficient understanding for the current task, it may be enough to just read the first function, or at least avoid reading every function—increased efficiency
- If a function call is encountered, we know where to start looking for its declaration—increased efficiency
- E.g. `NoughtsAndCrosses-MFV` from Discussion 5.1

Do One Thing (Revisited)

- Functions should do exactly one thing (also known as “cohesion”)
- It’s hard to know how many “things” a function is doing
- Good starting point—if you describe what the function using “and” or “or” then it’s probably doing more than one thing
- Functions that do one thing are probably small
- Large methods probably do more than one thing
- If a function only does one thing, it will be easier (more efficient) to:
  - understand it on first reading (compared with if it does multiple things)
  - remember what it does when encountering its use again
- no matter what size it is

Use Descriptive Names

- If the function does only one thing, its name should be that one thing it does
- Don't be afraid to use long names—e.g. `gameOverWithWinner()` and `gameOverWithDraw()`
- Functions that do one thing are probably easier to name

One Level Of Abstraction per Function

- Martin argues each function should have only one level of abstraction
- . . . but isn't too clear on what a "level of abstraction" is
- one possibly-useful heuristic is maximum of 2 levels of indentation—which is also something Martin advocates
- Functions with only one level of abstraction are probably small

  (or have a lot of regularity that can be exploited to reduce the requirements for short-term memory)

Why Arguments? Why Not Arguments?

- Functions without arguments significantly reduce their usefulness (recall the original reason for functions was to reduce code duplication)
- Martin argues “too many” argments creates (at least) two problems:
  - the arguments make the purpose of the function harder to understand
  - more arguments make it hard to test the function

  How to resolve this apparent contradiction?

Arguments (Parameters)

- According to Martin
  - zero arguments—Ideal
  - one argument—acceptable (but not flag arguments)
  - two arguments—barely acceptable
  - three arguments—avoid where possible
  - more than three arguments—what are you thinking!!
  - output arguments—a problem waiting to happen
- Why avoid arguments?
  - “We could pass [the StringBuffer] around as an argument rather than making it [a field], but then our readers would have had to interpret it each time they saw it.”(p111)
    - But doesn’t the same apply to fields?
  - “Arguments are even harder from a testing point of view.” (p112)
    - But doesn’t the same apply to fields? Perhaps even more so as there are fewer explicit indications as to where variation can occur

Reducing number of parameters


```
// static for simplicity - normally would not be static
private static  Grid _grid;
private static  IO _io;
public static void play(IO io) {
    _io = io;
    _grid = new Grid(io);
    _grid.initialiseBoard();
    _grid.display();
    char player = 'X';
    while (!playerMove(io, _grid, player)) {
        player = changePlayer(player);
    }
}
....
private static boolean gameOverWithDraw() {
    if (_grid.checkDraw()) {
        _io.println("Draw!");
        _io.println("Game over (draw)");
        return true;
    }
    return false;
}
```

- Fields exist for a purpose (to represent state of objects)
- Adding fields only to reduce the number of parameters to functions is almost certainly a bad idea
- Heuristic: are the fields needed by public function?
- What does the program comprehension model say about “multiple parameters”?

Output Arguments

- arguments whose value is changed and the change is visible to the caller

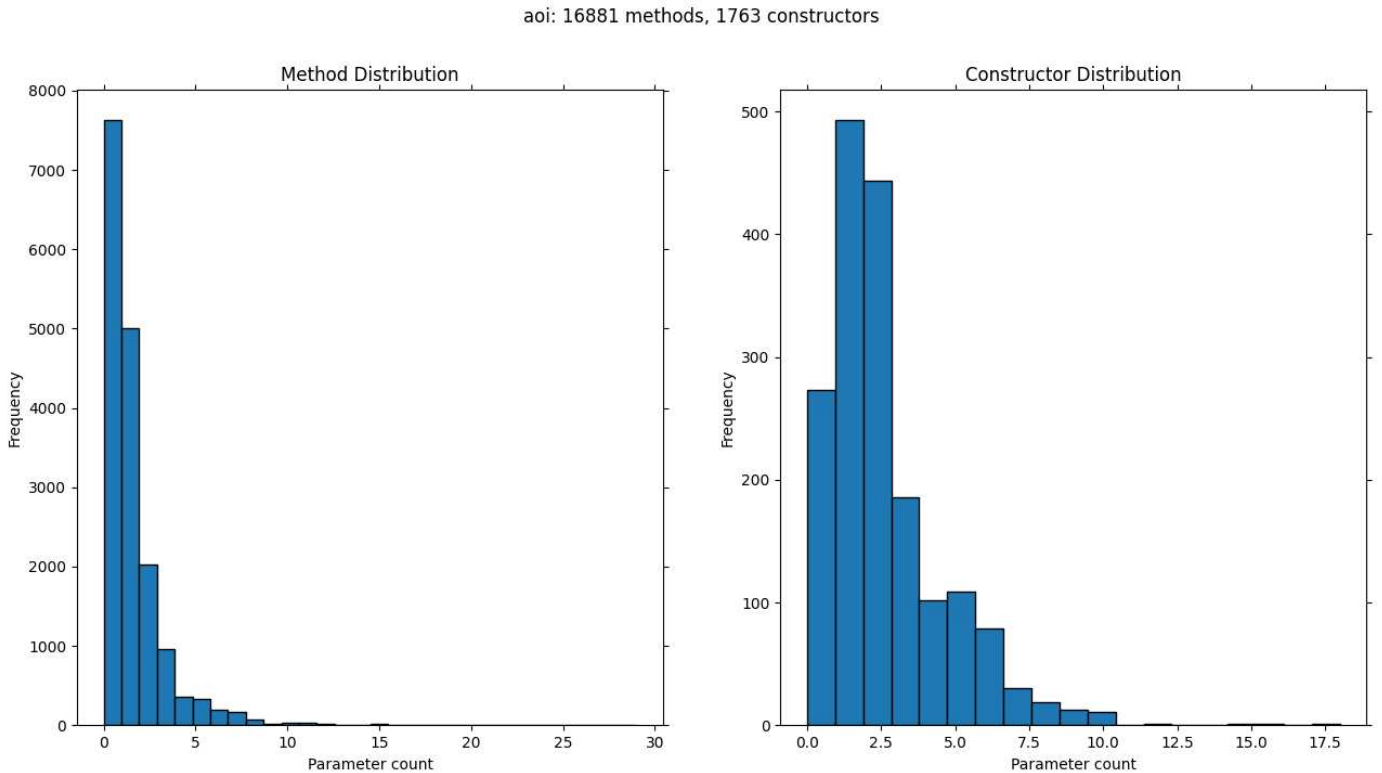
```
void replaceSpaceWithUnderscore(char[] chars) {
  for (int i = 0; i < chars.length; i++) {
    if (chars[i] == ' ') {
      chars[i] = '_';
    }
  }
}
```

- Martin argues this is confusing as we don't expect the argument value to be changed
- But it is easy to do

```
public void updateBoardWithMove(House houseChosenForMove) {
  // sets number of seeds to zero and returns the original value
  int numSeeds = houseChosenForMove.getAllSeeds();
  ...
}
```

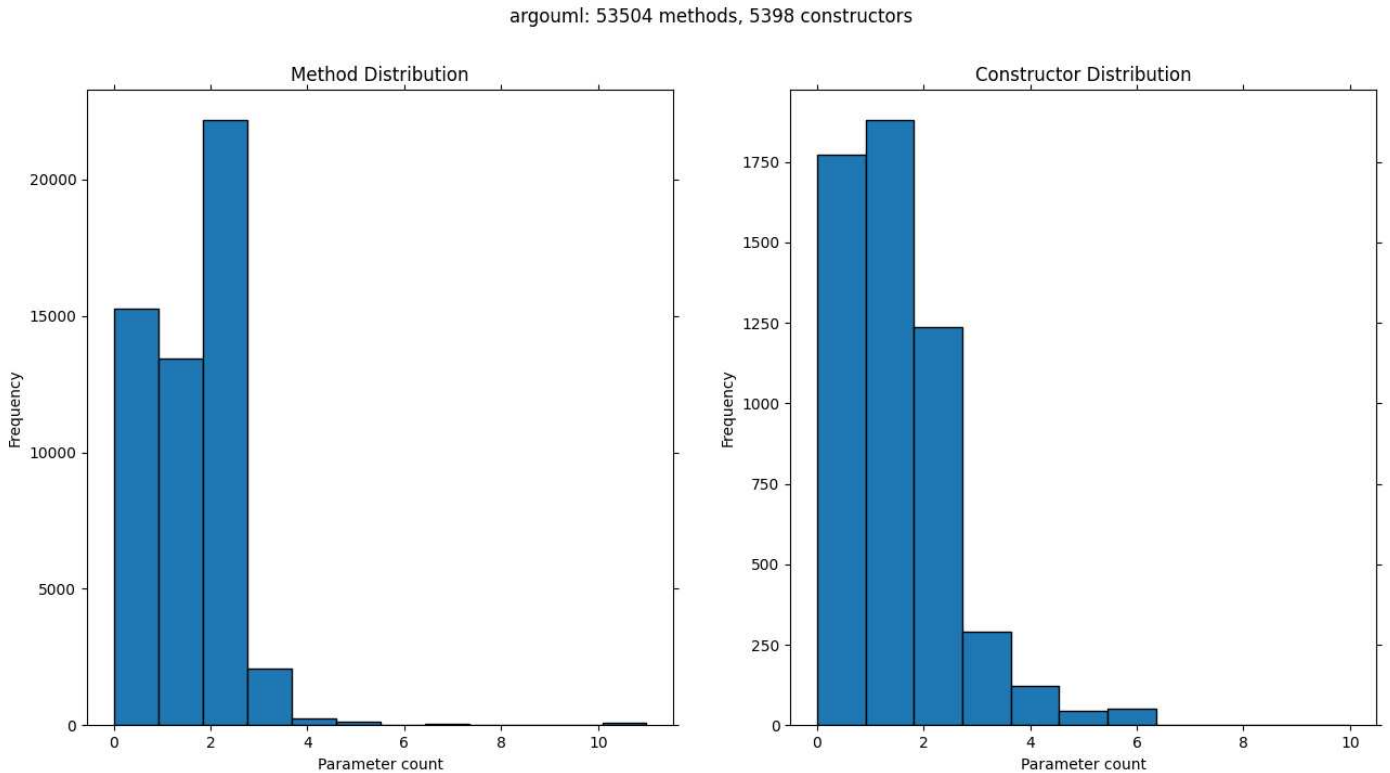
- (but the existence of the comment maybe points to a bigger problem)

Number of Parameters in Real Code



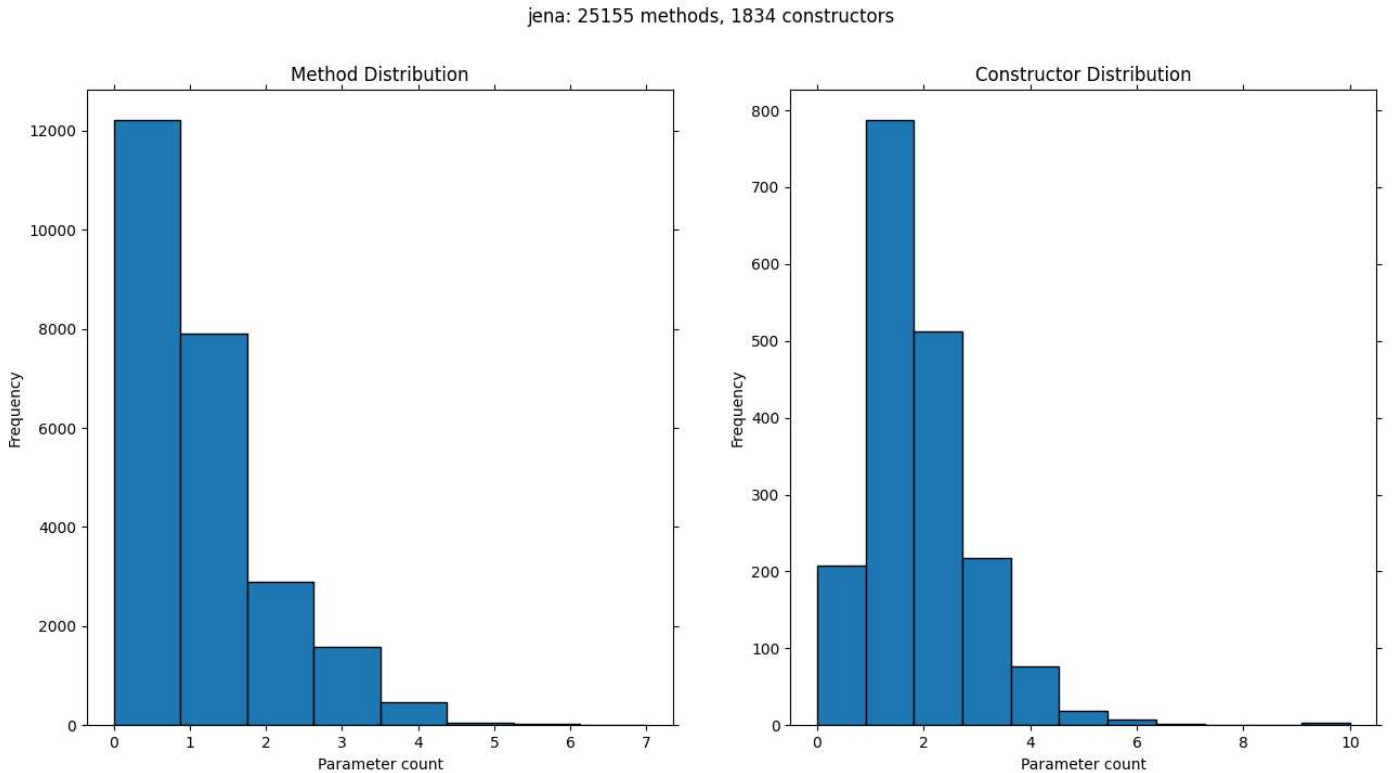
Art of Illusion. 3D Modelling and rendering. 385 classes, 111725 NCLOC
Saad Siddiqui and Seif Younes, SOFTENG Part IV project (received 0801 10 August 2021)

Number of Parameters in Real Code



ArgoUML. UML tool, 2560 classes, 192410 NCLOC
Saad Siddiqui and Seif Younes, SOFTENG Part IV project (received 0801 10 August 2021)

Number of Parameters in Real Code



Jena. Framework for Semantic Web and Linked Data applications, 1392 classes, 70948 NCLOC

Saad Siddiqui and Seif Younes, SOFTENG Part IV project (received 0801 10 August 2021)

How to reduce arguments (#2)

- Argument Objects Wrap some of the existing arguments into a class. E.g. (from Martin)

```
Circle makeCircle(double x, double y, double radius);
// or
Circle makeCircle(Point centre, double radius);
```

- Sometimes it is not obvious what the class should be

```
private boolean stillSearchingForFirstLetter(int nextIndex,
                                             int currentIndex,
                                             int limit) {
    return nextIndex >= 0 && nextIndex != currentIndex &&
           nextIndex < limit;
}
```

Possibly this is a sign this function was not meant to be

Don’t Repeat Yourself (DRY)

- Duplicate code generally can reduce maintainability. Not only does it mean having to read more code unnecessarily, but if you have to change one copy, then you probably need to change the other (if you don’t you’ve created a problem), so changing takes longer
- Removing duplicate code probably reduces the size of the methods involved

Class Exercise

- Examine `NoughtsAndCrosses-MFV.pdf`
- Investigate the feasibility of refactoring so that there are no methods longer than 4 lines
  - Begin by defining the top-level function in the “inverted pyramid” sense
  - Implement the first function call in the top-level function to be more concrete (but no longer than 4 lines)
  - Continue implementing the first function call until existing code can be used
  - Now refactor the next function that is longer than 4 lines.

Evaluation

- Does having all functions small really help with maintainability?
- There is a cost to making functions small. What is the benefit?
- When replacing a single large method with several smaller methods—
  - sometimes adding more functions reduces amount of code (removing duplication)
  - sometimes adding more functions increases amount of code (each function has some overhead)
- Martin's advice is a set of heuristics — so rather than apply the heuristics without question, understand why and so when they work

  $\Rightarrow$ what does the program comprehension model say?

Key Points

- Use of many small good functions can aid comprehension (at least)
  - because their names indicate the one thing that they do (self documenting code!)
  - often the functions with the lower levels of abstraction do not need to be considered, reducing the need to go to the external representation
- Refactor to improve use of functions
- We do not have a reliable way to determine the cost benefit ratio for most advice